

The design goals of JuliaDB are as follows:

- To have the capability to load multi-dimensional data sets faster with incremental support.
- The capability to index data and perform tasks such as aggregate, sort, filter and join.
- The capability to store the results and load them when required in a faster and more efficient way.
- The capability to use the parallelism to the maximum extent possible.

JuliaDB is built as an analytical database tool with all of the objectives mentioned earlier. The greatest advantage of JuliaDB is that it has been built all along in Julia itself, giving users the ability to compose queries with Julia code.

The fundamental components of JuliaDB are IndexedTable and NDParse data structures. Typed columns based storage makes the aggregation operations very fast.

Getting started with JuliaDB

Similar to any other package in Julia, JuliaDB can be added to your system with the following commands:

```
using Pkg
Pkg.add("JuliaDB")
```

```
using JuliaDB
```

A simple way to build a table is to use the following command:

```
t = table(rand(Bool, 10), rand(10))
```

A sample output is as shown below:

```
julia> t = table(rand(Bool, 10),
rand(10))
Table with 10 rows, 2 columns:
1      2
-----
true   0.94025
true   0.99059
false  0.135366
true   0.201484
```

Feature	JuliaDB	Pandas	xts	TimeArrays
Distributed computing	Yes	–	–	–
Data larger than memory	Yes	–	–	–
Multiple indexes	Yes	–	–	–
Index type(s)	Any	Built-ins	Time	Time
Value type(s)	Any	Built-ins	Built-ins	Any
Compiled UDFs	Yes	–	–	Yes

Table 1: A comparison of JuliaDB with other analytical databases

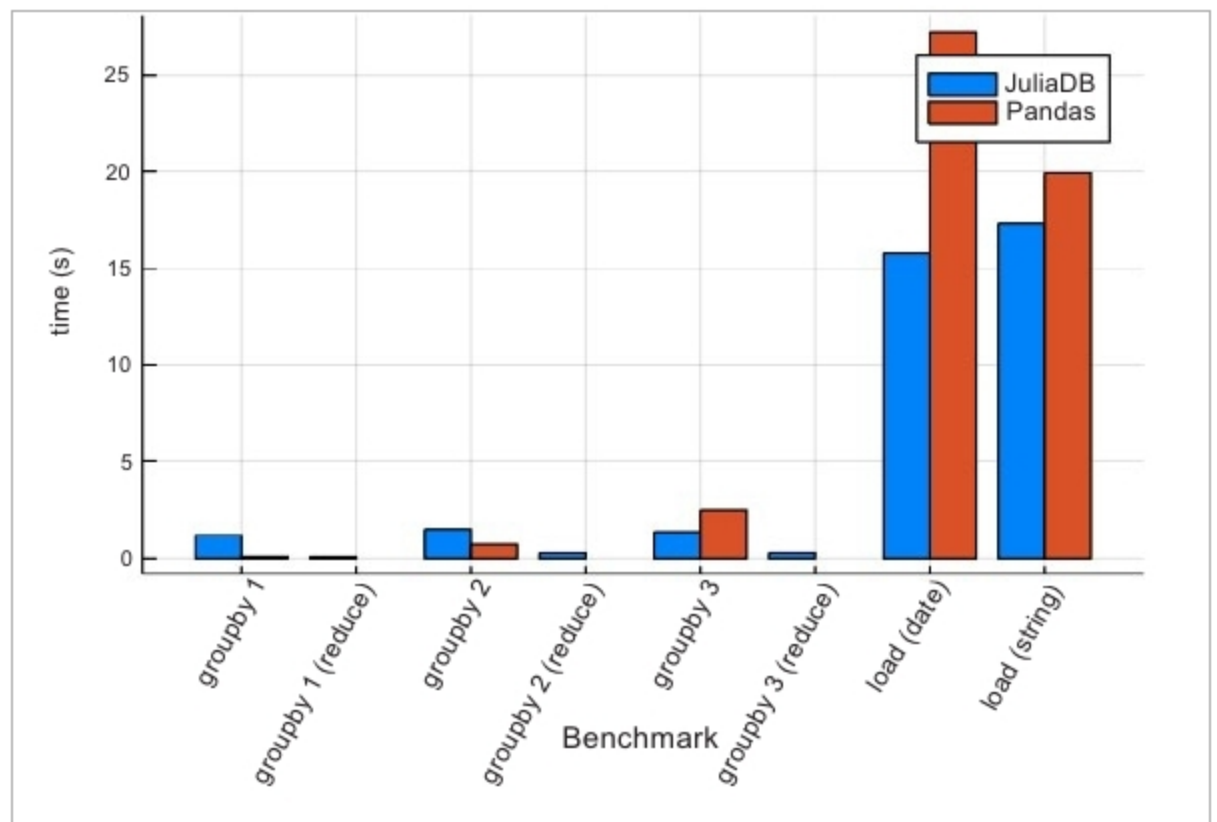


Figure 2: JuliaDB vs Pandas

```
true 0.454071
true 0.462337
false 0.299111
true 0.73977
true 0.485396
false 0.169812
```

Here, two columns have been built. The first column carries a Boolean value and the second one a random number.

Getting and loading data

As previously stated, JuliaDB is very good at handling CSV files. You can download the *hflights.csv* from <https://raw.githubusercontent.com/pieper/JuliaDBTutorial/master/hflights.csv>. Many examples in this article are based on this CSV file.

This CSV can be loaded with the following code snippet:

```
using JuliaDB
flights = loadtable("hflights.csv")
```

On successful loading of this table, you will get an output as shown in Figure 3.

It lists all the 21 columns and their data types.

Extracting data matching a condition

The rows that meet a particular condition can be extracted with the filter function. A sample filter function is shown below:

```
filter(i -> (i.Month == 10) &&
```